

CASOS DE ESTUDIO

DEPARTAMENTO DE CIENCIAS E INGENIERÍA DE LA COMPUTACIÓN

UNIVERSIDAD NACIONAL DEL SUR

KMC © 2026

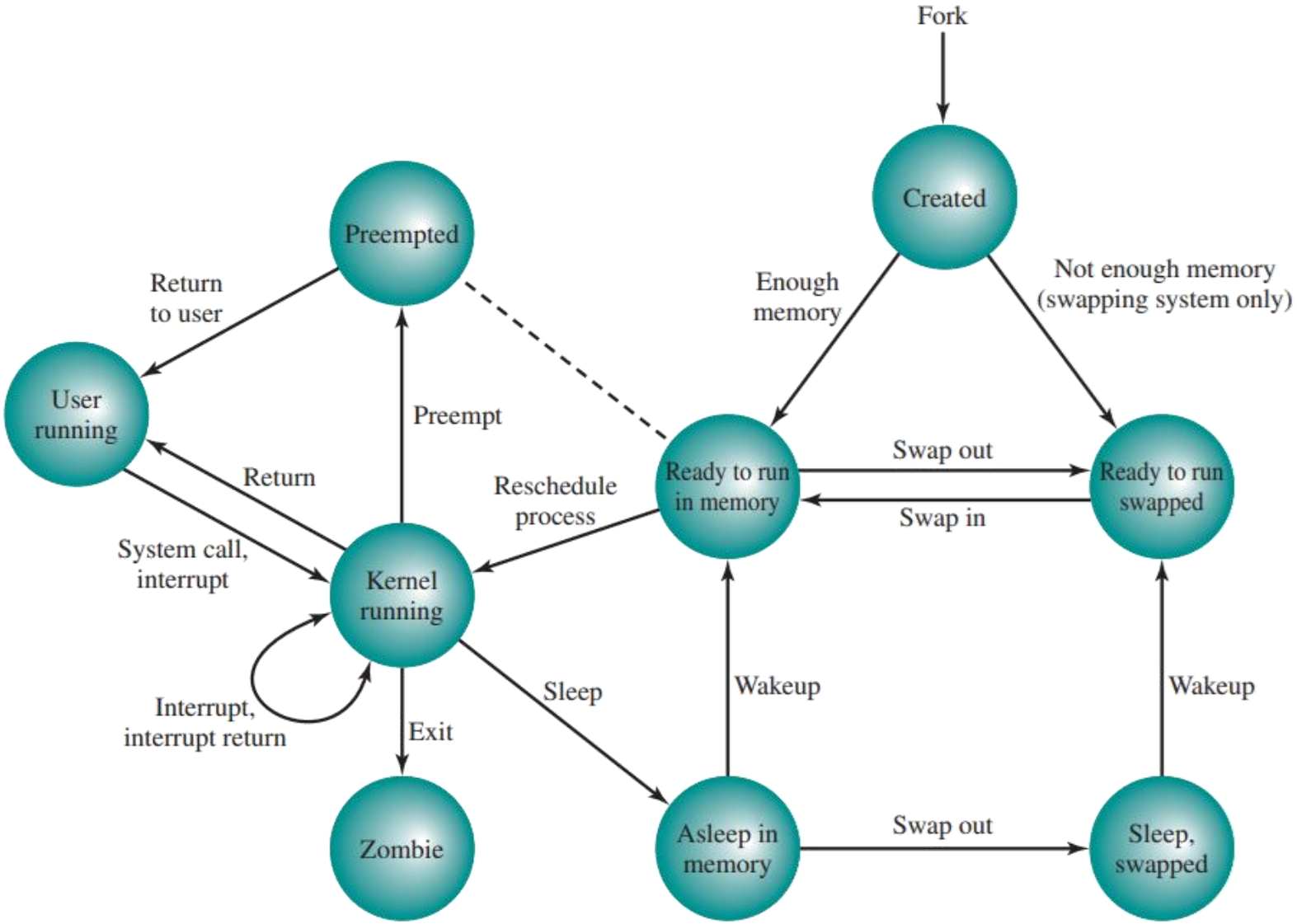
Agenda

1. UNIX
2. Linux
3. Solaris
4. Windows

Agenda

1. **UNIX**
2. Linux
3. Solaris
4. Windows

Unix – Diagrama Estado Procesos



Planificación tradicional DE UNIX

La planificación tradicional en UNIX emplea colas multinivel (los niveles se definen en bandas de prioridades) usando Round Robin en cada una de ellas.

$$P_j(i) = \text{Base}_j + \frac{CPU_j(i)}{2} + \text{nice}_j \qquad (1)$$

$$CPU_j(i) = \frac{CPU_j(i-1)}{2} \qquad (2)$$

(1) Es utilizada para ajustar dinámicamente la prioridad (producto del uso de CPU).

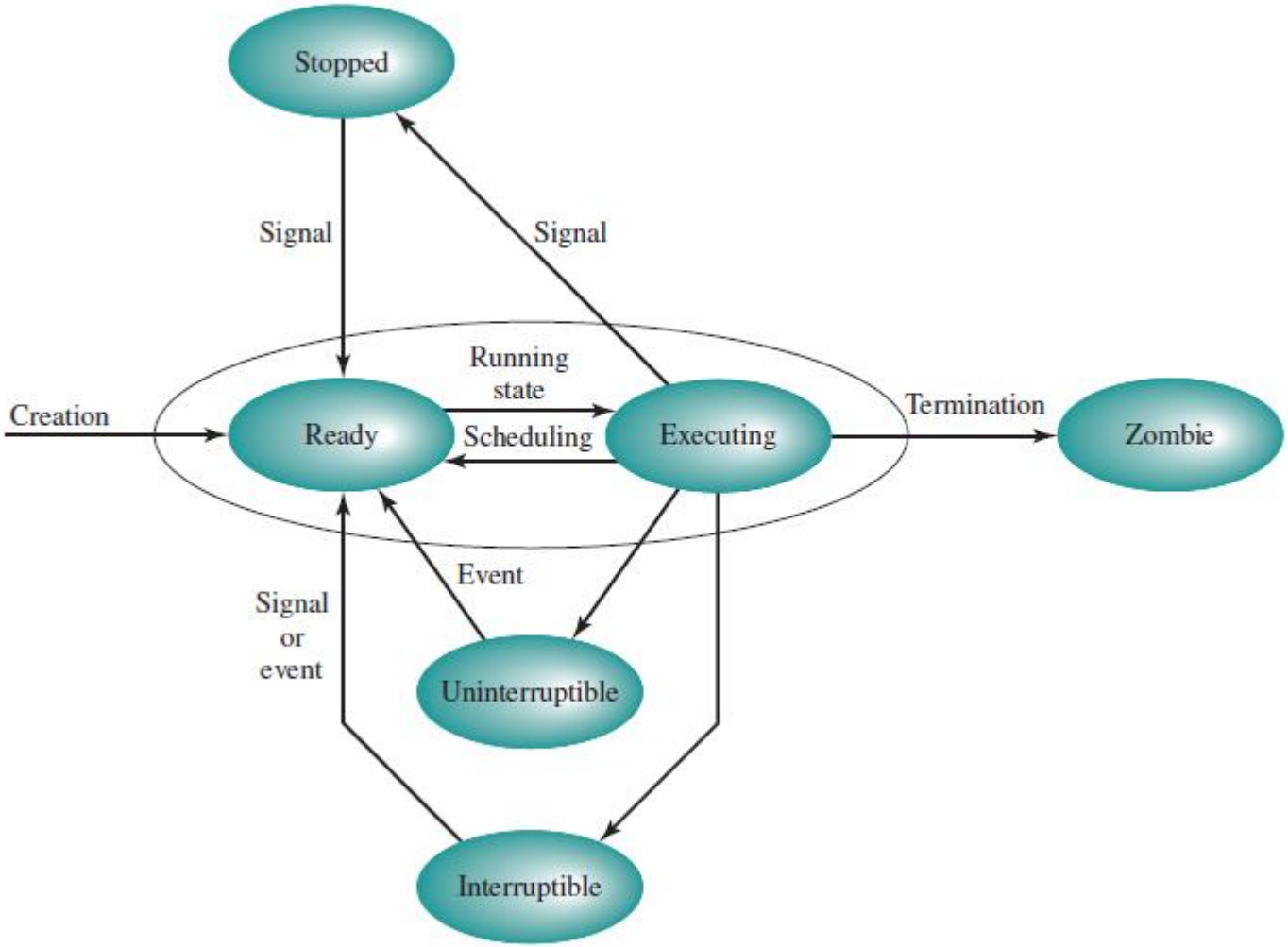
(2) Es usada para implementar el "envejecimiento" cuando el proceso espera. Así evita la inanición.

- CPU_j *(i)* = Mide la utilización del procesador por el proceso *j* en el intervalo *i*.
- P_j *(i)* = Prioridad del proceso *j* en el comienzo del intervalo *i*; valores bajos implican prioridades altas.
- Base_j = Prioridad base del proceso *j*.
- nice_j = Factor de ajuste controlable por el usuario

Agenda

1. UNIX
2. **Linux**
3. Solaris
4. Windows

Linux - Diagrama Estado Procesos



Planificación en Linux

- Anterior a la versión del kernel 2.5, corre una variación del algoritmo de planificación standard de UNIX
- Versión 2.5 propone un tiempo de planificación constante de orden *O*(1)
  - Apropiativo, basado en prioridades. Dos rangos de prioridades: tiempo compartido y tiempo real.
  - **Tiempo Real** rango entre 0 a 99 y valor de **nice** entre 100 y 140
  - Altas prioridades obtiene un mayor q
  - Una tarea está disponible para *running* mientras le quede tiempo en su intervalo de tiempo (**active**)
  - Si no le queda tiempo (**expired**), no está disponible para *running* hasta que todas las otras tareas utilicen sus intervalos de tiempo
  - Todas las tareas en runable se mantiene la información en la estructura de datos **runqueue** por CPU
    - Dos arreglos de prioridad (active, expired)
    - Tareas indexadas por prioridad
    - Cuando no hay más tareas en active, los arreglos se intercambian
  - Funcionó bien, pero los tiempos de respuesta deficientes para los procesos interactivos

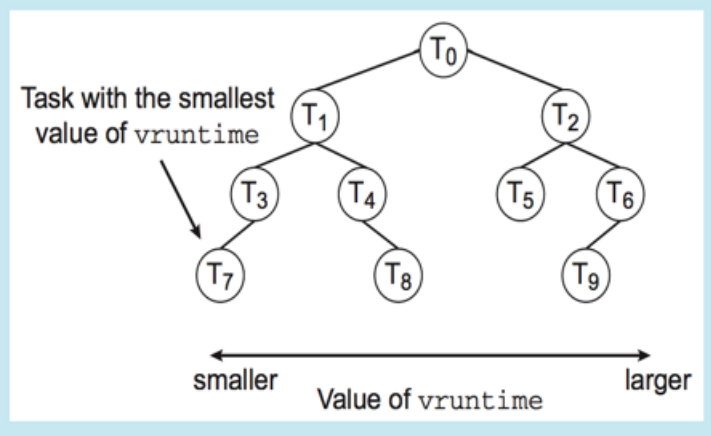
Planificación en Linux

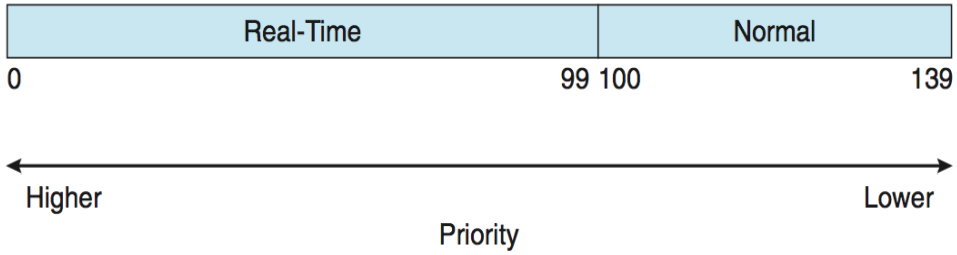
- Desde versión del kernel 2.6.23+, utiliza el algoritmo de ***Completely Fair Scheduler*** (CFS)
- Planifica clases
  - Cada uno tiene una prioridad específica.
  - El planificador selecciona la tarea con mayor prioridad en la clase de planificación más alta.
  - Se basa en la proporción de tiempo de CPU.
  - Incluye 2 clases de planificación, se pueden añadir otras.
    - Predeterminado.
    - Tiempo real.
- Quantum calculado en función del valor nice de -20 a +19.
  - Cuanto menor sea el valor, mayor será la prioridad.
  - Calcula la latencia objetivo: intervalo de tiempo durante el cual la tarea debe ejecutarse al menos una vez.
  - La latencia objetivo puede aumentar si, por ejemplo, aumenta el número de tareas activas.

Planificación en Linux -

kernel 2.6.23+ (cont)

- El planificador CFS mantiene el tiempo de ejecución virtual por tarea en la variable vruntime.
  - Se asocia con un factor de decaimiento basado en la prioridad de la tarea: una prioridad más baja tiene una tasa de decaimiento más alta.
  - La prioridad predeterminada normal produce un tiempo de ejecución virtual = tiempo de ejecución real.
- Para decidir la siguiente tarea que se ejecutará, el planificador elige la tarea con el tiempo de ejecución virtual más bajo.





Agenda

1. UNIX
2. Linux
3. **Solaris**
4. Windows

Solaris

Process

- Incluye el espacio de direcciones del usuario, stack y el PCB.

User-level Threads

- Unidad de ejecución creada por el usuario en el proceso.

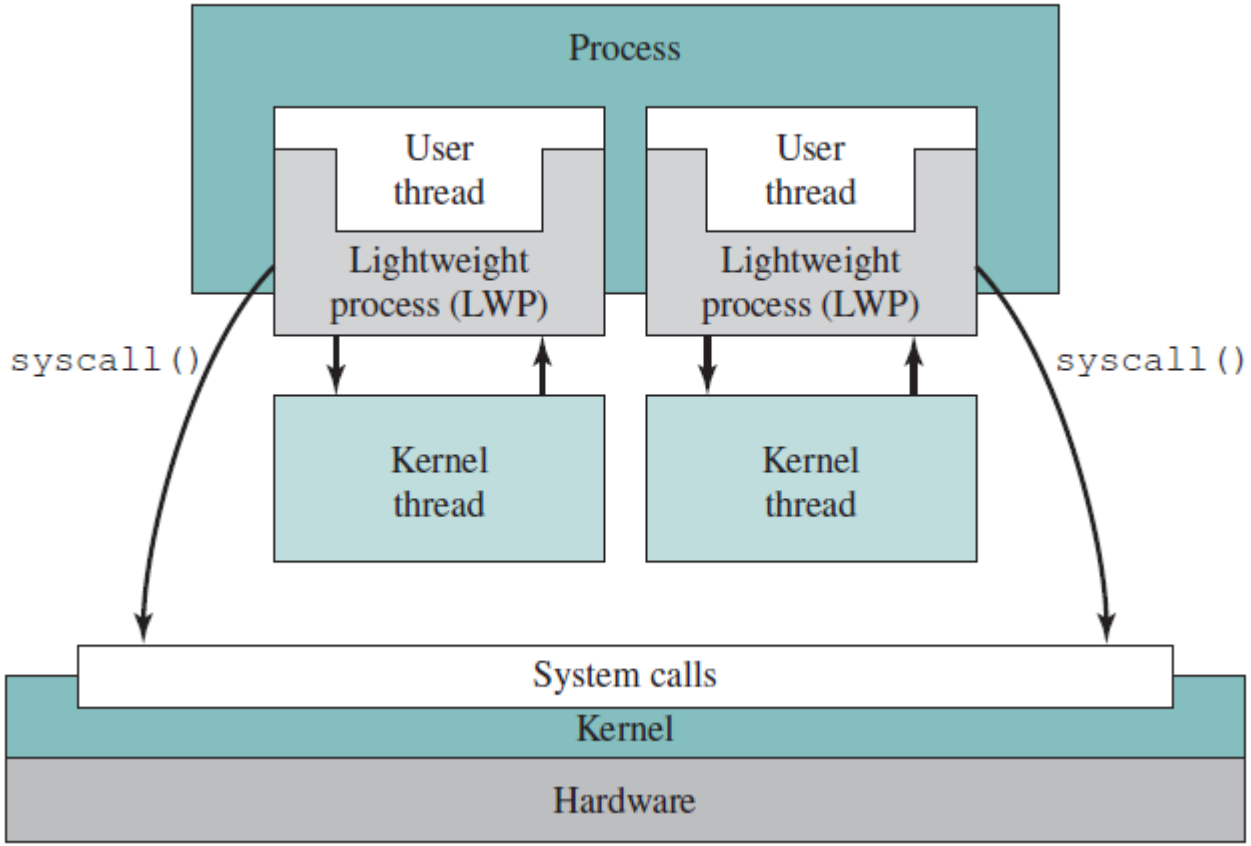
Lightweight Processes (LWP)

- Mapeo entre ULTs y kernel threads.

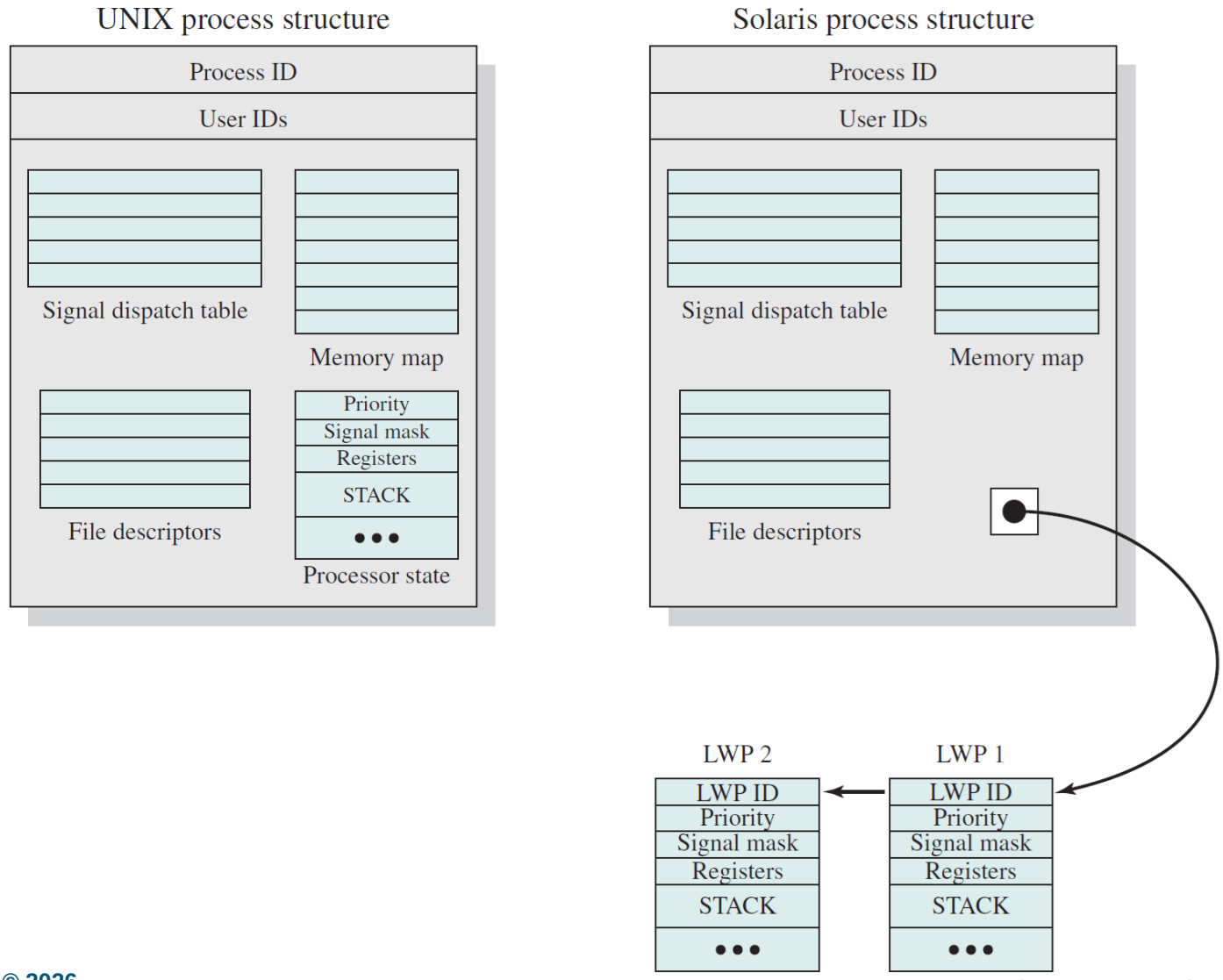
Kernel Threads

- Entidad fundamental que puede ser planificada y despachada para ejecutar en uno de los procesadores del Sistema.

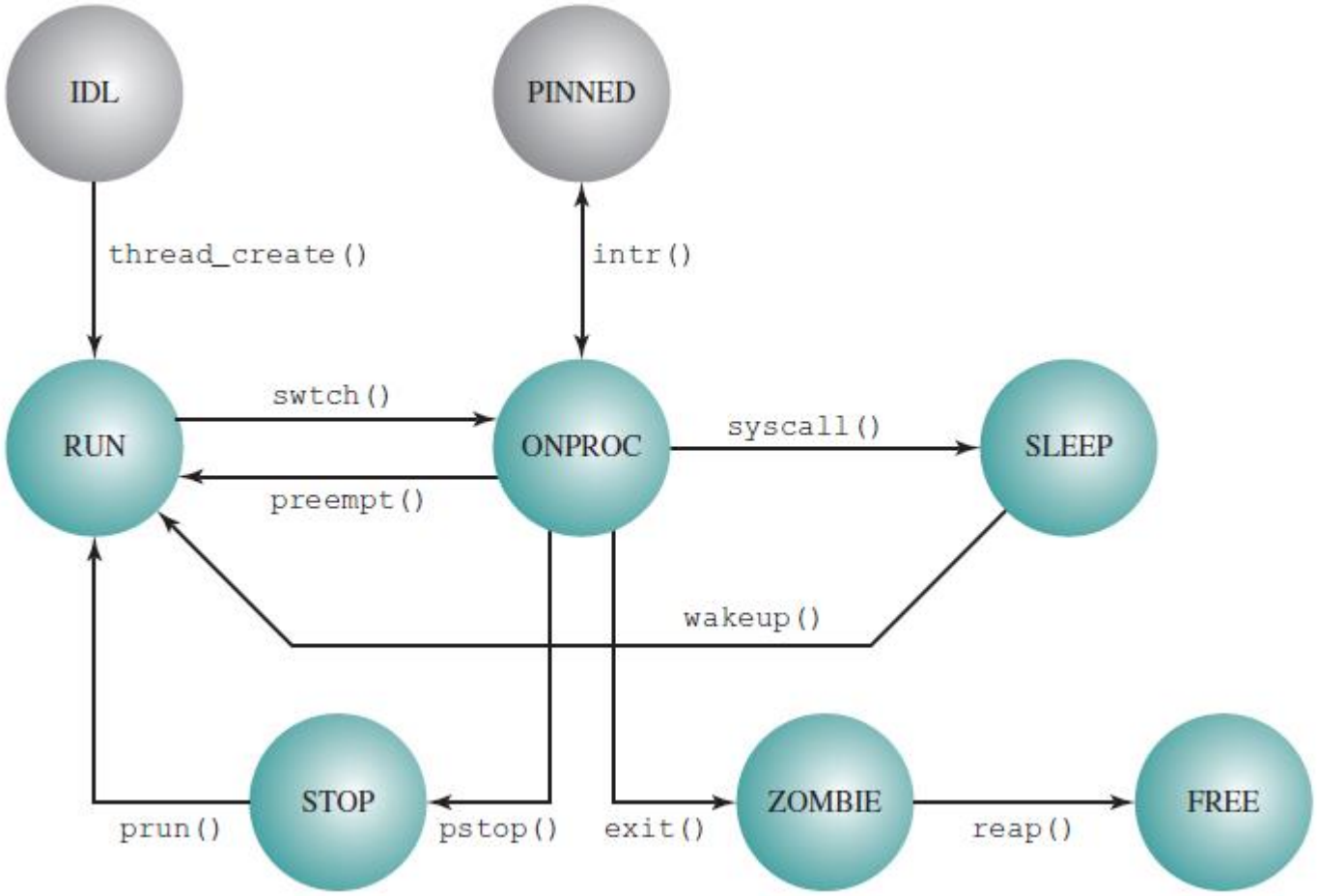
Solaris


Process
User thread
Lightweight process (LWP)
User thread
Lightweight process (LWP)
syscall()
syscall()
Kernel thread
Kernel thread
System calls
Kernel
Hardware

Solaris


UNIX process structure
Process ID
User IDs
Signal dispatch table
Memory map
File descriptors
Priority
Signal mask
Registers
STACK
Processor state
Solaris process structure
Process ID
User IDs
Signal dispatch table
Memory map
File descriptors
LWP 2
LWP 1
LWP ID
Priority
Signal mask
Registers
STACK

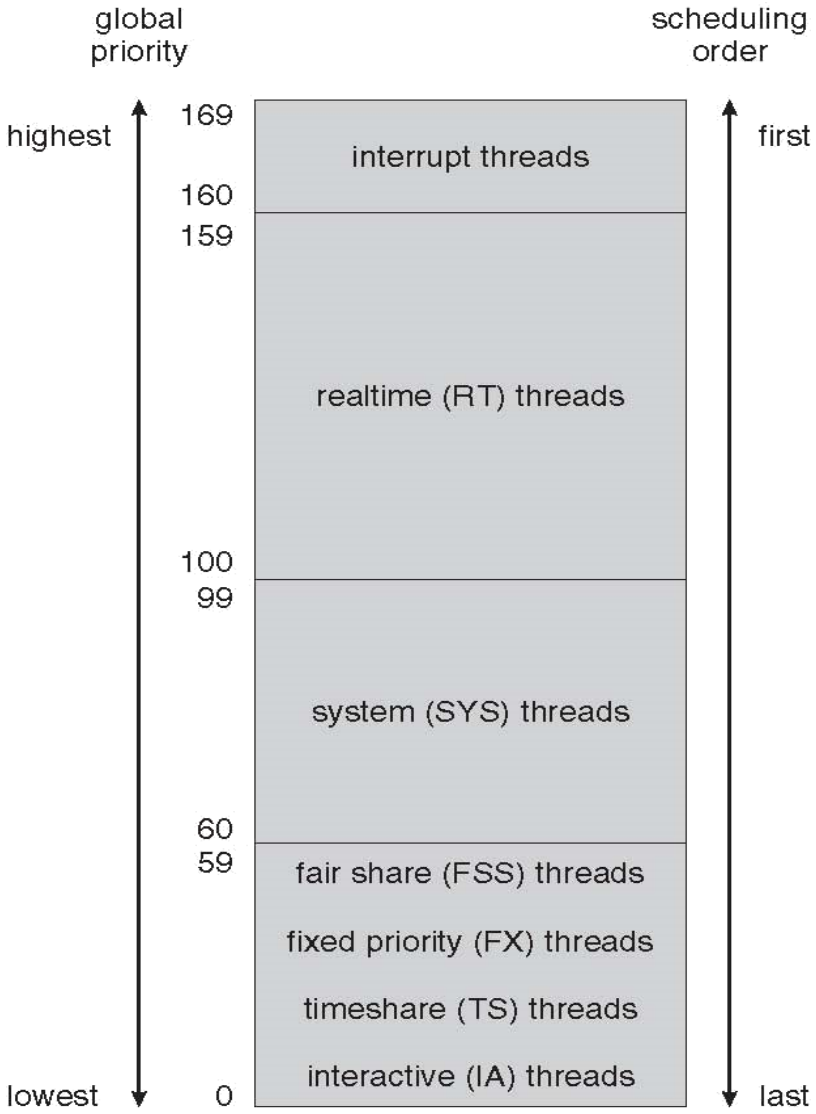
Solaris - Diagrama Estado Hilos



Solaris - Planificación

- Seis clases disponibles.
  - Tiempo compartido (predeterminado) (TS).
  - Interactivo (IA).
  - Tiempo real (RT).
  - Sistema (SYS).
  - Distribución equitativa (FSS).
  - Prioridad fija (FP).
- Un hilo determinado puede estar en una clase a la vez.
- Cada clase tiene su propio algoritmo de planificación.
- El tiempo compartido es una cola de retroalimentación multinivel.

Solaris - Planificación



Solaris - Planificación

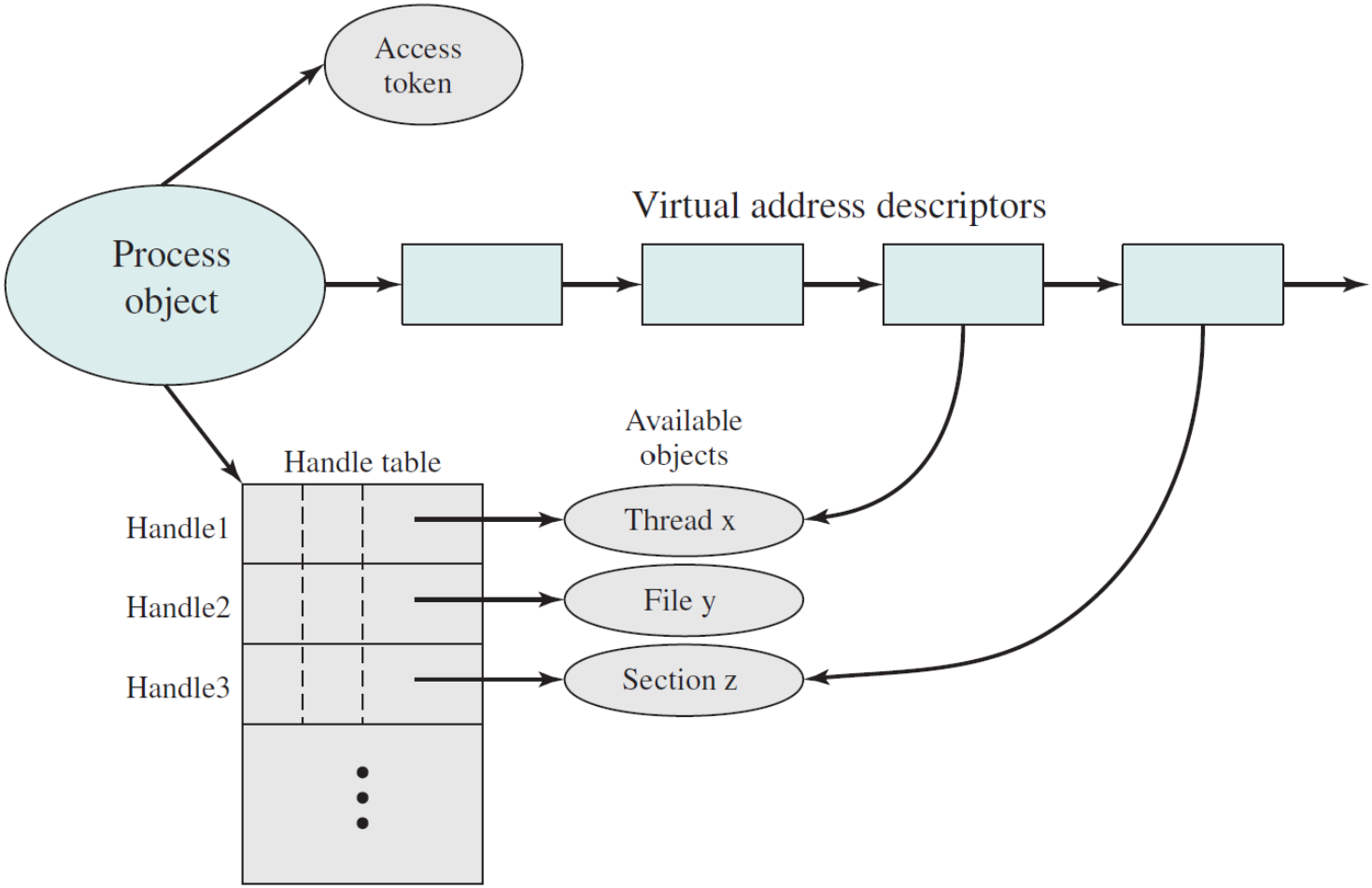
- El planificador convierte las prioridades específicas de cada clase en una prioridad global por hilo.
  - El hilo con mayor prioridad se ejecuta a continuación.
  - Se ejecuta hasta que (1) se bloquea, (2) utiliza el intervalo de tiempo, (3) es sustituido por un hilo de mayor prioridad.
  - Varios hilos con la misma prioridad se planifican mediante RR.

Agenda

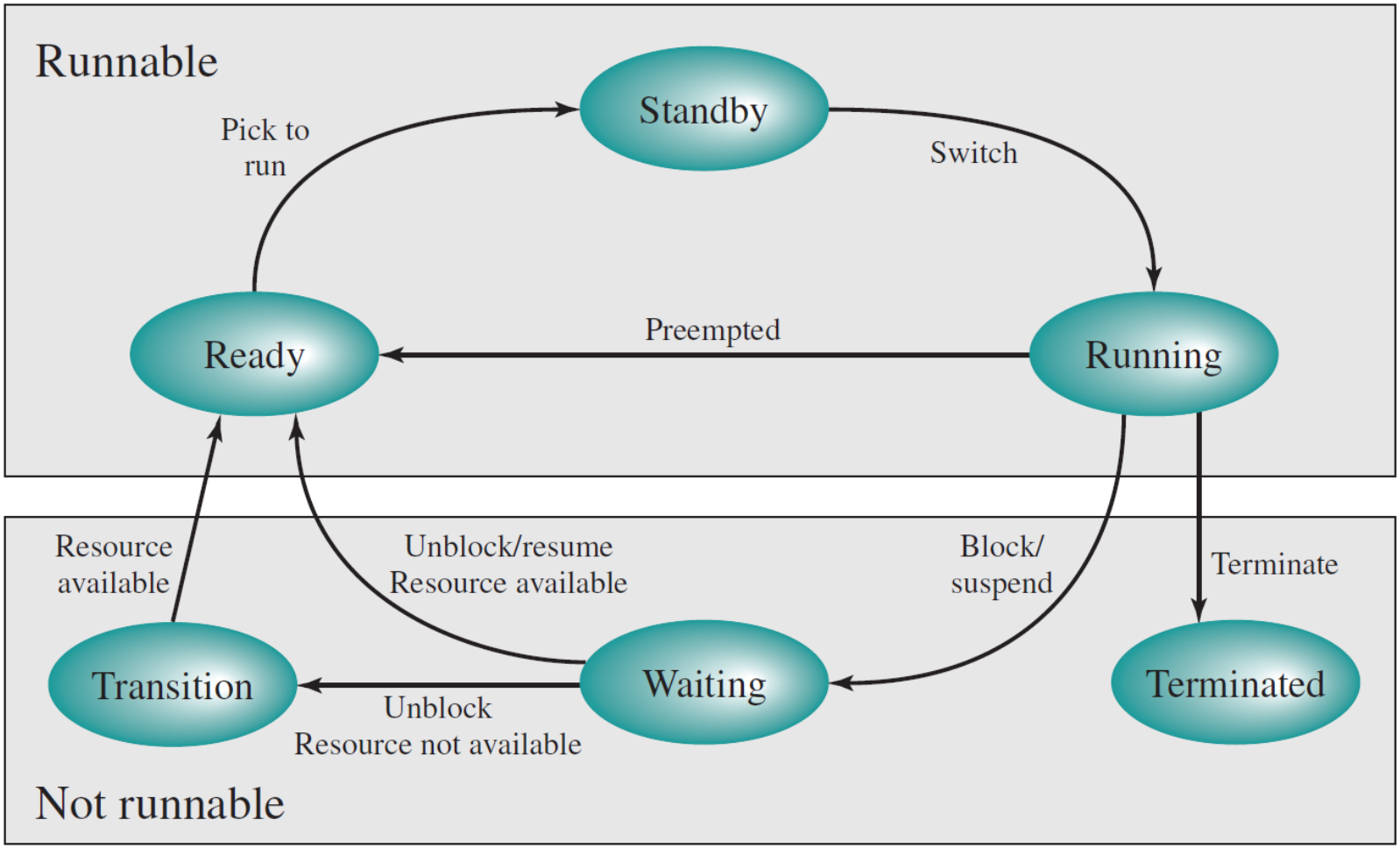
1. UNIX
2. Linux
3. Solaris
4. **Windows**

Windows

Un proceso en Windows y sus recursos



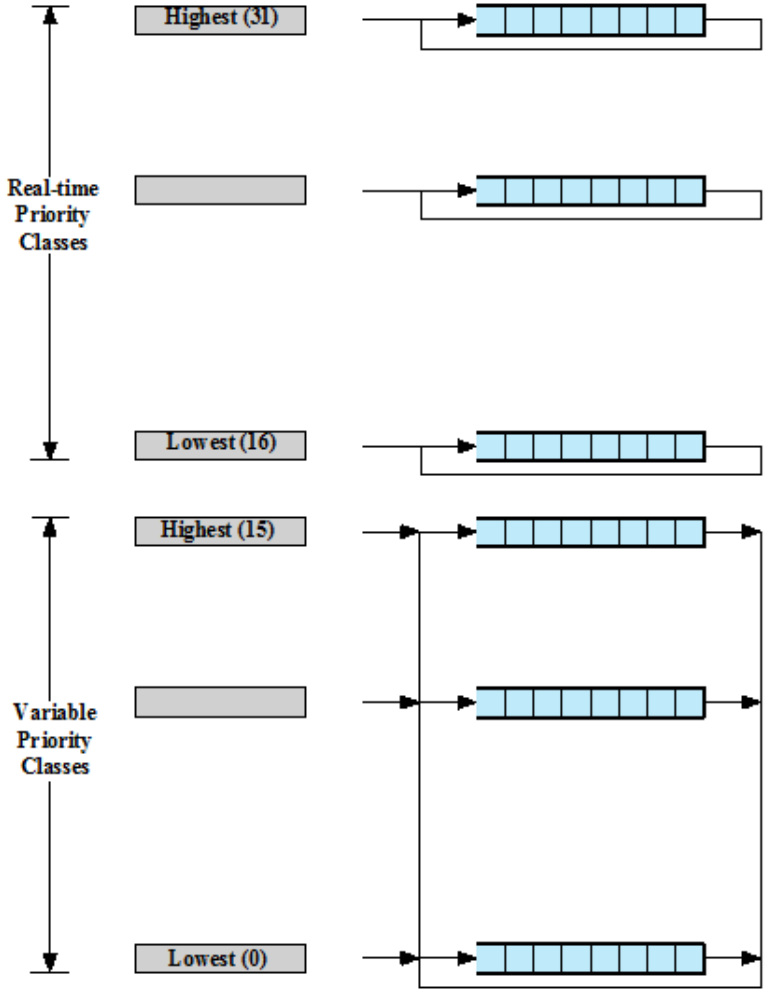
Windows – Diagrama de Estado Hilos



Windows - Planificación

- Windows utiliza una planificación apropiativa basada en prioridades.
- El hilo con mayor prioridad se ejecuta a continuación.
- El hilo se ejecuta hasta que (1) se bloquea, (2) utiliza el intervalo de tiempo, (3) es sustituido por un hilo con mayor prioridad.
- Los hilos en tiempo real pueden apropiar a los que no lo son.
- Esquema de prioridades de 32 niveles.
- La VARIABLE CLASS es 1-15, la REAL-TIME CLASS es 16-31.
- La prioridad 0 es el hilo de gestión de memoria.
- Una cola para cada prioridad.
- Si no hay ningún hilo ejecutable, se ejecuta el hilo inactivo (IDLE).

Windows - Planificación



Bibliografía:

- Silberschatz, A., Gagne G., y Galvin, P.B.; "Operating System Concepts", 7ma Edición 2009, 9na Edición 2012, 10ma Edición 2018.
- Stallings, W. "Operating Systems: Internals and Design Principles", Prentice Hall, 7ma Edición 2011, 8va Edición 2014, 9na Edición 2018.
- Tanenbaum, A.; "Modern Operating Systems", Addison-Wesley, 4ta. Edición 2014, 5th. Edition 2023.